

IF25-22017 PENGEMBANGAN APLIKASI MOBILE

Modul Belajar 10

# Testing, Dependency Injection, dan Debugging

Menulis unit test dan UI test dengan fake, merakit dependensi dengan Koin, dan membaca stack trace saat aplikasi crash

**Muhammad Habib Algifari, S.Kom., M.T.I.**

Pendamping Pertemuan 10. Tahun 2026

| Komponen      | Keterangan                                                                                                           |
|---------------|----------------------------------------------------------------------------------------------------------------------|
| Pertemuan     | 10                                                                                                                   |
| Capaian       | CPMK0502, menjelaskan konsep pemrograman lewat test yang menjelaskan perilaku kode dan dependensi yang tersusun rapi |
| Waktu belajar | sekitar 5 jam belajar mandiri, ditambah 2,5 jam di kelas                                                             |

## MODUL 10

## Peta belajar

---

Ikuti urutan ini dari atas ke bawah. Setiap bagian memakai apa yang sudah dibahas di bagian sebelumnya.

| Urutan | Bagian                                                |
|--------|-------------------------------------------------------|
| 1      | Tentang modul ini                                     |
| 2      | Masalah pembuka: fitur lama yang diam-diam rusak      |
| 3      | Langkah 1: Piramida test                              |
| 4      | Langkah 2: Test tinggal di commonTest                 |
| 5      | Langkah 3: Test pertama, Arrange, Act, Assert         |
| 6      | Langkah 4: Fake, Sumber tiruan untuk test             |
| 7      | Langkah 5: Menguji FormLaporanViewModel               |
| 8      | Langkah 6: Menguji AsistenViewModel saat gagal        |
| 9      | Langkah 7: UI test dengan runComposeUITest            |
| 10     | Langkah 8: Dependency injection dan modul Koin        |
| 11     | Langkah 9: Memasang Koin, dan modul uji               |
| 12     | Langkah 10: Debugging, membaca stack trace dan Logcat |
| 13     | Latihan mandiri                                       |
| 14     | Bawa ke proyekmu                                      |
| 15     | Rangkuman                                             |
| 16     | Cek pemahaman                                         |
| 17     | Bacaan lanjut dan persiapan minggu depan              |

---

## PEMBUKA

## Tentang modul ini

---

Modul ini menemani Pertemuan 10, pertemuan materi terakhir sebelum fase proyek kelompok dimulai di Pertemuan 11. Sampai Modul 9, LaporanKampus sudah punya lima layar, data dari server, penyimpanan offline, fitur platform, dan asisten AI. Semakin banyak fitur, semakin besar peluang fitur lama rusak diam-diam saat kita menambah fitur baru.

Modul ini terdiri dari empat bagian. Langkah 1 sampai 4 membahas **unit test** pertama dan **fake** yang menggantikan server. Langkah 5 sampai 7 membahas cara menguji **ViewModel dan layar**.

Langkah 8 dan 9 membahas **dependency injection** dengan Koin. Langkah 10 membahas **debugging**: membaca stack trace dan menyaring Logcat untuk crash yang lolos dari test.

### Setelah menyelesaikan modul ini, kalian bisa

| No | Kemampuan                                                                             | Dibahas di         |
|----|---------------------------------------------------------------------------------------|--------------------|
| 1  | Menulis unit test untuk aturan UiState dan memakai fake sebagai pengganti sumber data | Langkah 1 sampai 4 |
| 2  | Menguji ViewModel dengan runTest dan layar dengan runComposeUiTest                    | Langkah 5 sampai 7 |
| 3  | Merakit sumber data dan ViewModel dengan Koin, lalu menggantinya saat test            | Langkah 8 dan 9    |
| 4  | Membaca stack trace dan menyaring Logcat untuk menemukan baris yang salah             | Langkah 10         |

Capaian ini mendukung **CPMK0502**: menjelaskan konsep pemrograman, dalam bentuk test yang menjelaskan perilaku kode dan dependensi yang tersusun rapi. Indikator RPS untuk observasi minggu ini adalah test coverage minimal 60% dan DI yang terkonfigurasi.

### Yang perlu disiapkan

Proyek LaporanKampus dari Modul 9, terutama FormLaporanUiState, FormLaporanViewModel, AsistenViewModel, dan AsistenPalsu. Semua test di modul ini dijalankan di target desktop, jadi tidak butuh emulator.

#### **PENTING** Mengingat modul sebelumnya

Keputusan yang kita ambil sejak Modul 2 kini terbayar. ViewModel menerima Sumber dan AsistenLaporan lewat constructor (Modul 4, 6, dan 9), layar menerima ViewModel sebagai parameter (Modul 4), dan setiap sumber punya versi palsu (Modul 2, 6, dan 9). Tanpa keputusan itu, test di modul ini butuh internet kampus dan kunci API.

## PEMBUKA

## Masalah pembuka: fitur lama yang diam-diam rusak

Setiap minggu kita menambah fitur. Yang dicoba hanya fitur barunya, karena mengetes ulang lima layar satu per satu terlalu lama. Riwayat seperti ini bisa terjadi di kelompok mana saja:

| Waktu    | Yang terjadi                                                                                                                                           | Dicoba?                   |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Minggu 6 | Form mengirim laporan ke server                                                                                                                        | Dicoba manual, aman       |
| Minggu 9 | Tombol Pakai saran AI ditambahkan. Di commit yang sama, <code>isNotBlank</code> di <code>bisaKirim</code> ikut berubah menjadi <code>isNotEmpty</code> | Form tidak dicoba ulang   |
| Demo     | Laporan berjudul tiga spasi masuk ke admin fasilitas                                                                                                   | Ketahuan di depan penguji |

**TEBAK DULU** Coba pikirkan

Bagaimana memastikan kode tetap benar setiap kali menambah fitur, tanpa mengetes manual satu per satu?

**JAWABAN** Arah jawabannya

Jawaban yang sering muncul adalah lebih teliti saat review. Itu membantu, tetapi manusia lelah dan lupa, dan tidak ada yang mengetik tiga spasi saat mencoba sendiri. Yang kita butuhkan adalah **pemeriksaan yang ditulis sekali dan dijalankan berkali-kali** secara otomatis. Pemeriksaan seperti itu disebut test.

## LANGKAH 1

## Piramida test

| Jenis            | Jumlahnya | Yang diperiksa                                      | Kecepatan                                       |
|------------------|-----------|-----------------------------------------------------|-------------------------------------------------|
| UI test          | Sedikit   | Layar dinyalakan, tombol ditekan                    | Lebih lambat, mudah rusak saat tampilan berubah |
| Integration test | Beberapa  | Beberapa lapisan sekaligus, dibahas di Pertemuan 13 | Sedang                                          |

| Jenis     | Jumlahnya | Yang diperiksa                                         | Kecepatan          |
|-----------|-----------|--------------------------------------------------------|--------------------|
| Unit test | Banyak    | Satu kelas atau fungsi, tanpa layar dan tanpa jaringan | Hitungan milidetik |

Bentuknya seperti piramida: banyak unit test di bawah, sedikit UI test di atas. Makin ke bawah, makin banyak jumlahnya, makin cepat, dan makin mudah menunjuk baris yang salah. Bank Proyek meminta minimal **10 unit test dan 3 UI test** untuk proyek kelompok.

#### ANALOGI **Memeriksa bahan bangunan**

Memeriksa kekuatan setiap batu bata dan besi satu per satu jauh lebih murah dan cepat daripada menguji gedung yang sudah jadi. Kalau ada yang retak, langsung ketahuan bata yang mana. Unit test memeriksa bahan, UI test memeriksa gedung.

## LANGKAH 2

# Test tinggal di commonTest

composeApp/build.gradle.kts

```
kotlin {
    sourceSets {
        commonTest.dependencies {
            implementation(kotlin("test"))
            implementation(libs.kotlinx.coroutines.test)
            implementation(libs.koin.test)
            implementation(libs.compose.ui.test) // versi sama dengan Compose
        }
        Multiplatform {
            val desktopTest by getting
            desktopTest.dependencies {
                implementation(compose.desktop.currentOs)
            }
        }
    }
}
```

| Folder          | Isinya                      |
|-----------------|-----------------------------|
| commonMain      | Kode aplikasi               |
| commonTest      | Test untuk semua platform   |
| androidUnitTest | Test yang butuh API Android |

Di `libs.versions.toml`, tulis `compose-ui-test` dengan module `org.jetbrains.compose.ui:ui-test` dan `version.ref` ke versi Compose Multiplatform, sesuai dokumentasi Testing Compose Multiplatform

UI di [kotlinlang.org](https://kotlinlang.org). Kalau proyek memakai nama `jvmMain` untuk desktop, source set test-nya bernama `jvmTest`.

Jalankan semua test dengan perintah berikut. Target desktop menjalankan isi `commonTest` di JVM, jadi paling cepat dan tidak butuh emulator.

#### Terminal

```
./gradlew :composeApp:desktopTest
```

Kalian juga bisa menjalankan satu kelas atau satu test lewat ikon segitiga hijau di sebelah nama kelas atau fungsi di Android Studio.

### LANGKAH 3

## Test pertama, Arrange, Act, Assert

`commonTest/ui/FormLaporanUiStateTest.kt`

```
import kotlin.test.Test
import kotlin.test.assertFalse
import kotlin.test.assertTrue

class FormLaporanUiStateTest {
    @Test
    fun formKosongTidakBisaDikirim() {
        val state = FormLaporanUiState() // Arrange
        val hasil = state.bisaKirim      // Act
        assertFalse(hasil)               // Assert
    }

    @Test
    fun judulHanyaSpasiTidakBisaDikirim() {
        val state = FormLaporanUiState(judul = "  ", lokasi = "GKU 1 R201")
        assertFalse(state.bisaKirim)
    }

    @Test
    fun judulDanLokasiTerisiBisaDikirim() {
        val state = FormLaporanUiState(judul = "AC mati", lokasi = "GKU 1 R201")
        assertTrue(state.bisaKirim)
    }

    @Test
    fun sedangMengirimTidakBisaDikirimLagi() {
        val state = FormLaporanUiState(judul = "AC mati", lokasi = "GKU 1 R201",
        sedangMengirim = true)
        assertFalse(state.bisaKirim)
    }
}
```

| Bagian                          | Artinya                            |
|---------------------------------|------------------------------------|
| <code>@Test</code>              | Fungsi ini dijalankan sebagai test |
| <code>assertFalse(x)</code>     | Gagal kalau x bernilai true        |
| <code>assertTrue(x)</code>      | Gagal kalau x bernilai false       |
| <code>assertEquals(a, b)</code> | Gagal kalau a dan b berbeda        |

Setiap test mengikuti pola **Arrange, Act, Assert**: siapkan keadaan, lakukan satu aksi, lalu periksa hasilnya. Untuk aturan sederhana, Act dan Assert boleh digabung seperti test kedua.

Nama test ditulis sebagai kalimat perilaku. Saat test merah, namanya langsung memberi tahu aturan mana yang rusak, tanpa perlu membaca isinya.

#### ANALOGI **Memasak dan mencicipi**

Arrange seperti menyiapkan bahan, Act seperti memasak, dan Assert seperti mencicipi apakah rasanya sesuai resep. Satu test, satu masakan, satu kali mencicipi.

#### TEBAK DULU

Di commit minggu 9, bisaKirim memakai `judul.isEmpty()`. Kita jalankan empat test di atas. Berapa yang lulus, dan test mana yang gagal?

#### JAWABAN

**Tiga lulus, satu gagal**, yaitu `judulHanyaSpasiTidakBisaDikirim`. Tiga spasi tidak kosong, jadi `isEmpty()` bernilai true dan form bisa dikirim. `isNotBlank()` menganggap teks yang hanya berisi spasi sebagai kosong. Begitu diganti kembali ke `isNotBlank`, keempat test hijau.

## Coba di Kotlin Playground

Playground tidak menyediakan `kotlin.test`, jadi kita buat fungsi `cek` sendiri.

```
Main.kt
```

```

data class FormLaporanUiState(
    val judul: String = "",
    val lokasi: String = "",
    val sedangMengirim: Boolean = false
) {
    val bisaKirim get() = judul.isNotEmpty() && lokasi.isNotEmpty() && !sedangMengirim
}

fun cek(nama: String, syarat: Boolean) =
    println(("if (syarat) "LULUS " else "GAGAL ") + nama)

fun main() {
    cek("formKosongTidakBisaDikirim", !FormLaporanUiState().bisaKirim)
    cek("judulHanyaSpasiTidakBisaDikirim",
        !FormLaporanUiState(judul = " ", lokasi = "GKU 1 R201").bisaKirim)
    cek("judulDanLokasiTerisiBisaDikirim",
        FormLaporanUiState("AC mati", "GKU 1 R201").bisaKirim)
    cek("sedangMengirimTidakBisaDikirimLagi",
        !FormLaporanUiState("AC mati", "GKU 1 R201", true).bisaKirim)
}

```

#### Keluaran

```

LULUS formKosongTidakBisaDikirim
GAGAL judulHanyaSpasiTidakBisaDikirim
LULUS judulDanLokasiTerisiBisaDikirim
LULUS sedangMengirimTidakBisaDikirimLagi

```

Ganti kedua `isEmpty` menjadi `isNotBlank`, lalu jalankan lagi. Semuanya LULUS. Siklusnya selalu sama: test merah menunjuk aturan yang dilanggar, kita ubah kode, test menjadi hijau. Setelah hijau, test itu tetap tinggal di proyek dan menjaga aturan yang sama di setiap perubahan berikutnya.

#### LANGKAH 4

## Fake, Sumber tiruan untuk test

Untuk menguji ViewModel, kita tidak mau memanggil server sungguhan. Kita butuh pengganti yang cepat, bisa diatur, dan bisa diperiksa. Pengganti seperti itu disebut **fake**.

```
commonTest/data/SumberUji.kt
```

```

class SumberUji : Sumber<Laporan> {
    val ter kirim = mutableListOf<Laporan>()
    var gagal = false

    override suspend fun semua() = ter kirim.toList()
    override suspend fun ambil(id: Int) = ter kirim.first { it.id == id }
    override fun pantau(id: Int) = flowOf(ter kirim.first { it.id == id })
    override suspend fun kirim(bar u: Laporan): Laporan {
        if (gagal) error("internet kampus mati")
        val hasil = bar u.copy(id = ter kirim.size + 1)
        ter kirim += hasil
        return hasil
    }
}

```

| Sifat fake     | Di SumberUji                                                        |
|----------------|---------------------------------------------------------------------|
| Cepat          | Tanpa jaringan dan tanpa delay                                      |
| Bisa diatur    | <code>gagal = true</code> membuat kirim gagal                       |
| Bisa diperiksa | <code>ter kirim.size</code> menunjukkan berapa laporan yang dikirim |

Polanya sama dengan SumberLaporanPalsu dan AsistenPalsu. Bedanya, yang ini dibuat khusus untuk diperiksa di test. Karena FormLaporanViewModel hanya mengenal Sumber, ia tidak tahu sedang diberi server sungguhan atau SumberUji. Kalau ViewModel membuat SumberLaporanJaringan sendiri di dalamnya, kita tidak bisa menggantinya, dan test akan butuh internet kampus.

#### ANALOGI Manekin uji tabrak

Pabrik mobil tidak menguji sabuk pengaman dengan menabrakkan mobil berisi orang sungguhan. Mereka memakai manekin yang dilengkapi sensor, bisa diatur posisinya, dan bisa dibaca hasilnya setelah tabrakan. Fake adalah manekin itu.

Istilah lain yang akan kalian temui adalah **stub** dan **mock**. Untuk mata kuliah ini, fake sudah cukup.

**PENTING Kalau ViewModel kalian juga menyimpan draf**

FormLaporanViewModel di Modul 7 menerima DraftQueries langsung. Untuk test, ikuti saran di Modul 7: bungkus draf di balik interface kecil, lalu buat versi ujinya.

commonMain/data/PenyimpanDraf.kt

```
interface PenyimpanDraf {
    fun ambil(): Pair<String, String>?
    fun simpan(judul: String, lokasi: String)
    fun hapus()
}

class DrafUji : PenyimpanDraf {
    var isi: Pair<String, String>? = null
    override fun ambil() = isi
    override fun simpan(judul: String, lokasi: String) { isi = judul to lokasi }
    override fun hapus() { isi = null }
}
```

Versi aslinya memanggil DraftQueries di dalam fungsi-fungsi tersebut. Contoh test di modul ini ditulis dengan constructor satu parameter supaya ringkas. Tambahkan DrafUji() kalau ViewModel kalian butuh dua parameter.

**LANGKAH 5**

## Menguji FormLaporanViewModel

commonTest/ui/FormLaporanViewModelTest.kt

```

@OptIn(ExperimentalCoroutinesApi::class)
class FormLaporanViewModelTest {
    private val sumber = SumberUji()

    @BeforeTest fun mulai() = Dispatchers.setMain(StandardTestDispatcher())
    @AfterTest fun selesai() = Dispatchers.resetMain()

    @Test
    fun kirimBerhasilMengosongkanForm() = runTest {
        val vm = FormLaporanViewModel(sumber)
        vm.ubahJudul("AC mati")
        vm.ubahLokasi("GKU 1 R201")
        vm.kirim()
        advanceUntilIdle()
        assertEquals(1, sumber.terkirim.size)
        assertEquals("", vm.uiState.value.judul)
    }

    @Test
    fun kirimGagalTidakMenghapusIsian() = runTest {
        sumber.gagal = true
        val vm = FormLaporanViewModel(sumber)
        vm.ubahJudul("AC mati")
        vm.ubahLokasi("GKU 1 R201")
        vm.kirim()
        advanceUntilIdle()
        assertEquals("AC mati", vm.uiState.value.judul)
        assertNotNull(vm.uiState.value.pesan)
    }
}

```

| Bagian                                | Gunanya                                                 |
|---------------------------------------|---------------------------------------------------------|
| <code>Dispatchers.setMain(...)</code> | viewModelScope memakai dispatcher test, bukan Main asli |
| <code>@BeforeTest, @AfterTest</code>  | Dijalankan sebelum dan sesudah setiap test              |
| <code>runTest { }</code>              | Menjalankan test coroutine dengan waktu virtual         |
| <code>advanceUntilIdle()</code>       | Menjalankan semua coroutine yang masih antre            |

viewModelScope berjalan di `Dispatchers.Main`, yang tidak ada di test desktop. Karena itu Main diganti dispatcher test di BeforeTest, lalu dikembalikan di AfterTest. Karena Main sudah diganti, runTest memakai penjadwal yang sama, sehingga advanceUntilIdle ikut menjalankan coroutine di viewModelScope. Kelas ini butuh `@OptIn(ExperimentalCoroutinesApi::class)` untuk setMain dan StandardTestDispatcher.

ViewModel adalah kelas biasa. Cukup panggil fungsinya, lalu periksa state. Tidak ada layar yang perlu dinyalakan.

### ANALOGI Sutradara dan tombol putar

StandardTestDispatcher seperti sutradara yang menahan semua adegan di naskah sampai ia menekan tombol putar. `advanceUntilIdle` adalah tombol putar itu: semua adegan yang menunggu dijalankan sampai habis, baru penonton, yaitu `assert`, boleh menilai.

### TEBAK DULU

Baris `advanceUntilIdle()` kita hapus dari test pertama. Lulus atau gagal? Kalau gagal, `assert` yang mana dan apa pesannya?

### JAWABAN

**Gagal di `assert` pertama** dengan pesan `expected:<1> but was:<0>`.  `kirim()` hanya menaruh coroutine di antrean `StandardTestDispatcher`. Tanpa `advanceUntilIdle`, coroutine itu belum berjalan saat `assert` diperiksa, jadi belum ada laporan yang terkirim.

## LANGKAH 6

# Menguji `AsistenViewModel` saat gagal

Di Modul 9 kita berjanji menguji `AsistenViewModel` dengan asisten yang sengaja gagal.

```
commonTest/ui/AsistenViewModelTest.kt
```

```

class AsistenGagal : AsistenLaporan {
    override suspend fun sarankan(isi: String) =
        Result.failure<SaranLaporan>(IllegalStateException("kuota habis"))
}

@OptIn(ExperimentalCoroutinesApi::class)
class AsistenViewModelTest {
    @BeforeTest fun mulai() = Dispatchers.setMain(StandardTestDispatcher())
    @AfterTest fun selesai() = Dispatchers.resetMain()

    @Test
    fun awalnyaKosong() {
        val vm = AsistenViewModel(AsistenPalsu())
        assertEquals(AsistenUiState.Kosong, vm.ui.value)
    }

    @Test
    fun asistenBerhasilMenampilkanSaran() = runTest {
        val vm = AsistenViewModel(AsistenPalsu())
        vm.minta("AC di GKU 1 R201 netes ke meja")
        advanceUntilIdle()
        assertEquals(AsistenUiState.Saran, vm.ui.value)
    }

    @Test
    fun asistenGagalMenampilkanPesan() = runTest {
        val vm = AsistenViewModel(AsistenGagal())
        vm.minta("AC di GKU 1 R201 netes ke meja")
        advanceUntilIdle()
        assertEquals(AsistenUiState.Gagal, vm.ui.value)
    }
}

```

| Test     | Fake         | Keadaan yang diharapkan |
|----------|--------------|-------------------------|
| Awal     | AsistenPalsu | Kosong                  |
| Berhasil | AsistenPalsu | Saran                   |
| Gagal    | AsistenGagal | Gagal                   |

`assertIs<T>` memeriksa apakah sebuah nilai bertipe T. Kuota habis, timeout, dan JSON rusak semuanya menjadi `Result.failure`, jadi satu fake cukup untuk memastikan layar menampilkan Gagal dan tidak crash.

`delay(1000)` di `AsistenPalsu` tidak membuat test menunggu satu detik sungguhan, karena `runTest` memakai waktu virtual yang melompat langsung. Sampai di sini kita sudah punya empat test `UiState`, dua test `FormLaporanViewModel`, dan tiga test `AsistenViewModel`. Tambahkan satu lagi, misalnya bahwa judul lebih dari 40 karakter dipotong, dan sepuluh unit test sudah tercapai.

## LANGKAH 7

# UI test dengan runComposeUiTest

commonTest/ui/FormLaporanUiTest.kt

```

@OptIn(ExperimentalTestApi::class)
class FormLaporanUiTest {
    @Test
    fun kirimAktifSetelahFormDiisi() = runComposeUiTest {
        setContent {
            FormLaporanScreen(selesai = {}, vm = FormLaporanViewModel(SumberUji()))
        }
        onNodeWithText("Kirim").assertIsNotEnabled()
        onNodeWithText("Judul").performTextInput("AC mati")
        onNodeWithText("Lokasi").performTextInput("GKU 1 R201")
        onNodeWithText("Kirim").assertIsEnabled()
    }
}

```

Test ini memeriksa aturan bisaKirim dari sisi pengguna: tombol Kirim nonaktif saat form kosong, lalu aktif setelah judul dan lokasi diisi. `onNodeWithText("Judul")` menemukan kolom isian lewat label-nya. Layar diberi ViewModel dengan SumberUji, jadi tidak ada jaringan dan tidak butuh Koin. Parameter `vm` di `FormLaporanScreen` sejak Modul 4 yang membuat layar ini bisa diberi ViewModel apa saja.

Composable memang bisa diuji, tetapi butuh layar yang menyala dan lebih lambat. Karena itu logika ditaruh di ViewModel, dan UI test cukup memeriksa yang tampil.

## Cari, lakukan, periksa

| Langkah         | Fungsi                                                           | Contoh                                            |
|-----------------|------------------------------------------------------------------|---------------------------------------------------|
| Cari node       | <code>onNodeWithText</code> , <code>onNodeWithTag</code>         | <code>onNodeWithTag("tombolKirim")</code>         |
| Lakukan aksi    | <code>performClick</code> , <code>performTextInput</code>        | <code>performTextInput("AC mati")</code>          |
| Periksa         | <code>assertIsDisplayed</code> ,<br><code>assertIsEnabled</code> | <code>assertTextEquals("Laporan terkirim")</code> |
| Pastikan hilang | <code>assertDoesNotExist</code>                                  | Kartu Saran AI setelah Abaikan                    |

Setiap UI test mengikuti tiga langkah ini, sama dengan Arrange, Act, Assert. Teks tombol bisa berubah, misalnya Kirim menjadi Kirim laporan. Untuk node penting, beri `Modifier.testTag("tombolKirim")` lalu cari dengan `onNodeWithTag`. `testTag` tidak terlihat pengguna dan tidak dibacakan pembaca layar, jadi aman ditambahkan.

**CONTOH Dua UI test lain untuk proyek**

Layar daftar menampilkan laporan dari fake: isi SumberUji dengan dua laporan, lalu periksa kedua judulnya tampil dengan `assertIsDisplayed`. Layar AI menampilkan pesan saat gagal: beri AsistenViewModel dengan AsistenGagal, ketik deskripsi, tekan Minta saran, lalu periksa pesan gagalnya tampil.

**LANGKAH 8**

## Dependency injection dan modul Koin

### Merakit dependensi dengan tangan

Beginilah App.kt di banyak kelompok sekarang:

commonMain/App.kt

```
@Composable
fun LaporkampusApp() {
    val client = remember { buatClient() }
    val sumber = remember { SumberLaporanJaringan(LaporanApi(client)) }
    val nav = rememberNavController()
    NavHost(nav, startDestination = RuteBeranda) {
        composable<RuteForm> {
            val vm = viewModel { FormLaporanViewModel(sumber) }
            FormLaporanScreen(selesai = { nav.popBackStack() }, vm = vm)
        }
        // Detail, Asisten, Profil: dirakit dengan cara yang sama
    }
}
```

**Dependency injection** artinya objek menerima dependensinya dari luar, lewat constructor, bukan membuatnya sendiri. Kelas kita sudah begitu sejak Modul 4. Yang berantakan adalah **tempat merakitnya**. App.kt tahu semua kelas, dari HttpClient sampai setiap ViewModel. Setiap layar baru menambah baris remember dan viewModel. Kalau AsistenGemini butuh client dan kunci, App.kt juga yang harus tahu kuncinya.

**ANALOGI Penyelenggara acara**

Merakit dependensi dengan tangan seperti panitia wisuda yang menelepon sendiri tukang tenda, katering, sound system, dan fotografer, lalu memberi tahu masing-masing siapa bekerja dengan siapa. Koin seperti penyelenggara acara: panitia cukup menulis daftar kebutuhan sekali, dan penyelenggara yang menghubungkan semua pihak.

## Modul Koin

commonMain/di/ModulAplikasi.kt

```
import org.koin.core.module.dsl.viewModel
import org.koin.dsl.module

val dataModule = module {
    single { buatClient() }
    single { LaporanApi(get()) }
    single<Sumber<Laporan>> { SumberLaporanJaringan(get()) }
    single<AsistenLaporan> { AsistenPalsu() }
}

val uiModule = module {
    viewModel { FormLaporanViewModel(get()) }
    viewModel { AsistenViewModel(get()) }
    viewModel { (id: Int) → DetailViewModel(id, get()) }
}
```

| Kata          | Artinya                                    |
|---------------|--------------------------------------------|
| module { }    | Satu kelompok definisi cara membuat objek  |
| single { }    | Satu objek untuk seluruh aplikasi          |
| factory { }   | Objek baru setiap kali diminta             |
| viewModel { } | ViewModel yang ikut umur layar             |
| get()         | Koin mencari objek bertipe yang dibutuhkan |

Baca dari atas ke bawah. HttpClient dibuat sekali. LaporanApi butuh HttpClient. Sumber butuh LaporanApi. ViewModel butuh Sumber. `get()` mencari berdasarkan tipe parameter constructor.

`single<Sumber<Laporan>>` penting: yang diminta ViewModel adalah interface, bukan kelas jaringan. AsistenPalsu dipakai dulu, lalu diganti AsistenGemini saat kunci siap, cukup di satu baris itu. DetailViewModel butuh id dari rute, jadi id dikirim sebagai parameter. Kalau DetailViewModel kalian juga menerima Pemberitahu dari Modul 8, tambahkan `get()` kedua dan daftarkan Pemberitahu di modul platform pada Langkah 9.

### TEBAK DULU

Definisi Sumber kita ubah dari `single` menjadi `factory`, memakai SumberLaporanPalsu. Pelapor mengirim laporan dari form, lalu membuka detailnya. Apakah laporan baru itu ditemukan?

**JAWABAN**

**Tidak ditemukan.** Dengan factory, FormLaporanViewModel dan DetailViewModel masing-masing mendapat SumberLaporanPalsu baru dengan daftarnya sendiri. Laporan tersimpan di objek milik form, sedangkan detail membaca objek lain. Dengan single, semua layar berbagi satu sumber. factory cocok untuk objek kecil tanpa data bersama, misalnya validator.

**ANALOGI Dispenser dan gelas**

single seperti dispenser air di ruang jurusan: satu untuk semua, dan kalau galonnya diganti, semua orang minum air yang sama. factory seperti gelas plastik sekali pakai: setiap orang mendapat gelas baru, dan isi gelas satu orang tidak terlihat di gelas orang lain.

**LANGKAH 9**

## Memasang Koin, dan modul uji

### Di aplikasi

commonMain/App.kt

```
import org.koin.compose.KoinApplication

@Composable
fun App() {
    KoinApplication(application = {
        modules(dataModule, uiModule)
    }) {
        LaporanKampusApp()
    }
}
```

commonMain/ui/FormLaporanScreen.kt

```
import org.koin.compose.viewmodel.koinViewModel

@Composable
fun FormLaporanScreen(
    selesai: () → Unit,
    vm: FormLaporanViewModel = koinViewModel()
) { /* isi sama seperti sebelumnya */ }
```

**Pustaka**

koin-core

**Isinya**

module, single, factory, get

| Pustaka                | Isinya                       |
|------------------------|------------------------------|
| koin-compose           | KoinApplication              |
| koin-compose-viewmodel | viewModel dan koinViewModel  |
| koin-test              | KoinTest untuk menguji modul |

Semua pustaka dari grup `io.insert-koin`, dengan versi yang disamakan. Hands-on P11 memakai Koin seri 4.0. KoinApplication memulai Koin dari composable paling atas. App.kt sekarang tidak lagi berisi remember untuk Sumber atau viewModel dengan lambda. Untuk layar detail yang butuh id, tulis `koinViewModel<DetailViewModel> { parametersOf(id) }`.

**Isi layar tidak berubah. Yang berubah hanya cara memperoleh ViewModel.**

#### **PENTING Objek khusus platform**

Pemberitahu dari Modul 8 dibuat berbeda di tiap platform. Caranya, App menerima satu modul tambahan dari titik masuk, misalnya `App(modulPlatform = module { single<Pemberitahu> { PemberitahuDesktop() } })` di main.kt desktop, lalu App memasukkannya ke `modules(dataModule, uiModule, modulPlatform)`. Biarkan IDE menambahkan import, dan cocokkan dengan dokumentasi Koin untuk versi yang kalian pakai.

## Modul uji menggantikan modul data

commonTest/di/ModulAplikasiTest.kt

```
val modulUji = module {
    single<Sumber<Laporan>> { SumberUji() }
    single<AsistenLaporan> { AsistenGagal() }
}

class ModulAplikasiTest : KoinTest {
    @BeforeTest fun mulai() {
        startKoin { modules(uiModule, modulUji) }
    }
    @AfterTest fun selesai() = stopKoin()

    @Test
    fun formViewModelDirakitKoin() {
        val vm = get<FormLaporanViewModel>()
        assertFalse(vm.uiState.value.bisaKirim)
    }
}
```

uiModule sama persis dengan yang dipakai aplikasi. Hanya modul data yang diganti dengan modulUji. Test ini memastikan modul kita lengkap: kalau ada definisi yang lupa ditulis, `get()` gagal di sini, bukan saat aplikasi dibuka di depan penguji.

`stopKoin()` wajib dipanggil di `AfterTest`. Tanpa itu, test berikutnya gagal dengan `KoinAppAlreadyStartedException`. Untuk unit test `ViewModel` biasa, tetap lebih sederhana tanpa `Koin`, langsung `FormLaporanViewModel(SumberUji())` seperti Langkah 5.

#### LANGKAH 10

## Debugging, membaca stack trace dan Logcat

Test memberi tahu **ada** yang salah. Debugging mencari **di mana**. Untuk crash yang lolos dari test, kita perlu membaca stack trace.

### Kode yang akan crash

Jalankan kode ini di Kotlin Playground. Baris kosong sengaja dipertahankan supaya nomor barisnya sama dengan stack trace di bawah.

File.kt

```
data class LaporanDto(val id: Int, val prioritas: String)
data class Laporan(val id: Int, val prioritas: Int)

fun LaporanDto.keLaporan() = Laporan(id, prioritas.toInt())

fun muat(dto: LaporanDto): Laporan = try {
    dto.keLaporan()
} catch (e: NumberFormatException) {
    throw IllegalStateException("Gagal memuat ${dto.id}", e)
}

fun main() {
    val dariServer = listOf(LaporanDto(1, "2"),
        LaporanDto(7, "tinggi"), LaporanDto(9, "1"))
    for (dto in dariServer) println("Siap: ${muat(dto)}")
}
```

Server mengirim prioritas sebagai teks. Biasanya isinya angka, tetapi satu laporan berisi tinggi. Fungsi `muat` menangkap error itu, lalu melempar error baru dengan pesan yang lebih jelas, sambil membawa error aslinya sebagai argumen kedua. Kasus ini nyata: API publik sering tidak konsisten, dan bug seperti ini baru muncul saat data sungguhan masuk.

#### TEBAK DULU

Apa yang tercetak sebelum program berhenti? Lalu, baris nomor berapa yang harus diubah supaya ketiga laporan tercetak?

**JAWABAN**

Hanya Siap: Laporan(id=1, prioritas=2) yang tercetak. Program berhenti di laporan 7, dan laporan 9 tidak pernah dimuat. Baris yang diubah adalah **baris 4**, fungsi keLaporan, bukan baris 9 tempat throw. Banyak yang menjawab baris 9 karena itu yang disebut paling atas. Bagian berikut menjelaskan kenapa jawabannya baris 4.

**Membaca stack trace****Keluaran, baris panjang dipecah**

```
Siap: Laporan(id=1, prioritas=2)
Exception in thread "main"
  java.lang.IllegalStateException: Gagal memuat 7
    at FileKt.muat(File.kt:9)
    at FileKt.main(File.kt:15)
Caused by: java.lang.NumberFormatException:
  For input string: "tinggi"
    at java.lang.Integer.parseInt(Integer.java:...)
    at FileKt.keLaporan(File.kt:4)
    at FileKt.muat(File.kt:7)
    ... 2 more
```

| Bagian                                     | Artinya                                                      |
|--------------------------------------------|--------------------------------------------------------------|
| Baris paling atas                          | Error terluar dan pesannya, sering hanya pembungkus          |
| Caused by                                  | Penyebab aslinya. Kalau ada beberapa, baca yang paling bawah |
| Baris pertama FileKt di bawah<br>Caused by | Kode kita sendiri, tempat memperbaiki                        |
| Baris java, android, kotlinox              | Kode pustaka yang ikut terseret, lewati dulu                 |
| ... 2 more                                 | Sisa barisnya sama dengan trace di atas, jadi tidak diulang  |

Urutan bacanya: lihat baris paling atas untuk tahu jenis error, turun ke Caused by untuk tahu penyebabnya, lalu lewati baris java dan cari baris pertama dari kode kita sendiri. Di sini itu `FileKt.keLaporan` di `File.kt` baris 4.

Perbaikannya: `prioritas.toIntOrNull() ?: 3`. Artinya, kalau teksnya bukan angka, pakai prioritas bawaan 3. Coba di Playground, lalu ganti tinggi dengan teks kosong dan perhatikan apakah masih berjalan.

**ANALOGI Laporan kecelakaan**

Stack trace seperti laporan kecelakaan lalu lintas. Baris atas menyebut kejadiannya: mobil menabrak pagar. Caused by menyebut penyebab akarnya: rem blong karena kabel rem putus. Memperbaiki pagar tidak menyelesaikan masalah. Yang harus diperbaiki adalah kabel remnya.

## Stack trace di Android: Logcat

Di Android, stack trace muncul di jendela **Logcat** Android Studio. Baris yang berasal dari paket kita sendiri berwarna biru dan bisa diklik untuk langsung membuka berkasnya. Urutan bacanya sama: atas, Caused by, lalu baris dari paket sendiri.

Untuk melacak urutan kejadian, tambahkan log dengan awalan yang sama:

```
commonMain/ui/DetailViewModel.kt

fun muat(id: Int) {
    println("LaporKampus muat id=$id")
    viewModelScope.launch {
        val hasil = sumber.ambil(id)
        println("LaporKampus status=${hasil.status}")
    }
}
```

println dari commonMain masuk ke Logcat dengan tag System.out. Awalan yang sama di setiap pesan membuatnya mudah dicari dengan filter berikut:

| Filter Logcat       | Yang tampil                                    |
|---------------------|------------------------------------------------|
| package:mine        | Hanya aplikasi dari proyek yang sedang terbuka |
| level:ERROR         | Hanya error dan yang lebih parah               |
| is:crash            | Hanya crash aplikasi                           |
| message:LaporKampus | Pesan yang memuat kata itu                     |

Filter bisa digabung, misalnya `package:mine is:crash`. Log berguna saat kita ingin tahu urutan kejadian, misalnya apakah data dari jaringan datang sebelum atau sesudah data dari database. Jangan mencetak data pribadi pengguna di log, dan hapus log yang berisik sebelum demo. Cara memakai breakpoint untuk menghentikan program di baris tertentu kita bahas di Pertemuan 12.

### ANALOGI Fitur cari di grup chat

Logcat tanpa filter seperti grup chat angkatan dengan ribuan pesan. Filter seperti fitur cari: tampilkan hanya pesan dari satu orang, atau hanya pesan yang memuat kata tugas. Awalan LaporKampus di setiap log adalah kata kunci yang kalian sepakati supaya mudah dicari.

## PRAKTIK

## Latihan mandiri

Ada tiga latihan di folder P11 Testing dan DI Hands-on, di dalam commonTest/.../latihan. Ketiganya diamati, bersama laporan coverage.

| Latihan | Berkas               | Yang dilatih                                                                            |
|---------|----------------------|-----------------------------------------------------------------------------------------|
| 1       | CounterTest.kt       | Tiga test untuk increment, decrement di nol, dan reset dengan pola Arrange, Act, Assert |
| 2       | SessionLoggerTest.kt | FakeClockProvider dengan TestTimeSource, jam palsu yang hanya maju saat disuruh         |
| 3       | KoinModuleTest.kt    | startKoin dengan appModule, inject Counter dan SessionLogger, lalu stopKoin             |

Selama TODO di latihan lain belum diganti, desktopTest akan gagal di kelas itu. Jalankan satu kelas lewat ikon Run di Android Studio, atau tambahkan `--tests` dengan nama kelasnya di perintah Gradle.

### CONTOH Petunjuk latihan 1

Satu test, satu aturan, seperti Langkah 3. Untuk decrement di nol, siapkan counter bernilai 0, panggil decrement, lalu pastikan nilainya tetap 0, bukan -1.

### CONTOH Petunjuk latihan 2

Waktu asli membuat test kadang lulus kadang gagal, tergantung kecepatan laptop. TestTimeSource dari kotlin.time adalah jam palsu: waktunya hanya maju saat kalian memanggil += dengan durasi tertentu. Itu sama dengan prinsip fake di Langkah 4, hanya yang dipalsukan adalah jam.

### CONTOH Petunjuk latihan 3

Polanya sama dengan ModulAplikasiTest di Langkah 9. Jangan lupa stopKoin di AfterTest. Yang selesai duluan, tulis test bisaKirim dan FormLaporanViewModel di proyek kelompok.

## Mengukur coverage

**Coverage** adalah persentase baris kode yang dijalankan oleh test. Indikator RPS minggu ini meminta minimal 60%. Untuk target JVM, coverage bisa diukur dengan fitur Run with Coverage di Android

Studio, atau dengan plugin Kover dari JetBrains yang membuat laporan HTML. Ikuti cara yang tercantum di README hands-on P11, lalu tunjukkan angka coverage saat diamati.

#### **PENTING Coverage bukan tujuan**

Coverage 100% tidak berarti kode bebas bug. Test yang menjalankan baris tanpa memeriksa hasilnya dengan assert tetap menaikkan coverage, tetapi tidak menjaga apa pun. Yang penting adalah setiap test benar-benar memeriksa satu aturan.

## Yang diamati

| Level       | Skor          | Yang terlihat                                                                              |
|-------------|---------------|--------------------------------------------------------------------------------------------|
| Sangat baik | 85 sampai 100 | Coverage 60% atau lebih, Koin terkonfigurasi, dan bisa menjelaskan fungsi fake di test-nya |
| Baik        | 70 sampai 84  | Coverage 40 sampai 59% dan Koin berjalan                                                   |
| Cukup       | 55 sampai 69  | Ada test yang lulus, coverage di bawah 40% atau Koin belum berjalan                        |
| Kurang      | di bawah 55   | Belum ada test yang berjalan                                                               |

Observasi dikerjakan individu dengan bobot 4% dari nilai akhir, untuk **CPMK0502**. Buktinya: push latihan ke repository latihan pribadi, lalu kirim tautan commit di LMS sebelum kelas selesai. Saat berkeliling, saya akan meminta kalian menjelaskan fungsi fake di test kalian.

## Kalau error

| Gejala atau pesan                                       | Penyebab                                     | Yang dicoba                                  |
|---------------------------------------------------------|----------------------------------------------|----------------------------------------------|
| NotImplementedError: An operation is not implemented    | TODO() di berkas latihan belum diganti       | Lengkapi TODO, atau jalankan satu kelas saja |
| expected: but was:                                      | Coroutine di viewModelScope belum berjalan   | Panggil advanceUntilIdle() sebelum assert    |
| Error Main dispatcher, atau test ViewModel kadang gagal | viewModelScope memakai Dispatchers.Main asli | Dispatchers.setMain di BeforeTest            |
| KoinAppAlreadyStartedException                          | startKoin dua kali tanpa stopKoin            | Panggil stopKoin() di AfterTest              |
| NoDefinitionFoundException                              | Modul berisi definisinya belum didaftarkan   | Tambahkan modul itu di modules()             |
| Test kadang lulus, kadang gagal                         | Bergantung pada waktu asli                   | Pakai jam palsu seperti TestTimeSource       |
| Unresolved reference ke API Android di commonTest       | commonTest memanggil kode khusus platform    | Pindahkan test itu ke androidUnitTest        |

README hands-on masih menulis nama lama `NoBeanDefFoundException`. Versi Koin yang lebih baru menampilkan `NoDefinitionFoundException` untuk masalah yang sama.

#### PRAKTIK

## Bawa ke proyekmu

Mulai dari aturan yang pernah rusak di proyek kalian, karena test paling berharga adalah yang menjaga bug lama agar tidak kembali. Bagi kerja: satu anggota fake dan test `ViewModel`, satu anggota UI test, satu anggota modul Koin.

1. **Fake untuk setiap sumber.** Sumber data dan fitur AI tema kalian punya versi uji yang bisa diatur gagal.
2. **Sepuluh unit test.** `UiState` dan `ViewModel`: aturan simpan, berhasil, gagal, dan memuat.
3. **Tiga UI test.** Form, daftar, dan layar AI dengan `runComposeUiTest`.
4. **Koin.** Modul data dan ui, layar memakai `koinViewModel`, dan modul uji untuk test.

#### CONTOH Contoh dari `LaporKampus`

`FormLaporanUiStateTest` berisi empat test, `FormLaporanViewModelTest` dua, `AsistenViewModelTest` tiga, ditambah satu test pemotongan judul, sehingga genap sepuluh unit test. Lalu ada tiga UI test untuk form, daftar, dan layar AI. Semuanya lulus dengan `desktopTest` dalam hitungan detik.

## Checkpoint minggu 10

| No | Yang diselesaikan                                                                                                       |
|----|-------------------------------------------------------------------------------------------------------------------------|
| 1  | Koin terpasang: modul data dan ui, layar mengambil <code>ViewModel</code> dengan <code>koinViewModel</code>             |
| 2  | Sepuluh unit test untuk <code>UiState</code> dan <code>ViewModel</code> dengan fake, tanpa jaringan dan tanpa kunci API |
| 3  | Tiga layar diuji dengan <code>runComposeUiTest</code> , semuanya lulus                                                  |

Push ke repository kelompok. README memuat perintah menjalankan test dan tangkapan layar `desktopTest` yang lulus. Checkpoint tidak masuk nilai, tetapi menjadi bekal tes lisan di pertemuan 12 sampai 16. Minggu depan test ini dijalankan otomatis di setiap push, jadi pastikan lulus di laptop semua anggota.

## PENUTUP

# Rangkuman

---

Test adalah pemeriksaan yang ditulis sekali dan dijalankan berkali-kali. **Piramida test** berisi banyak unit test yang cepat di bawah dan sedikit UI test di atas. Test tinggal di `commonTest` dan dijalankan dengan `desktopTest`.

Setiap test mengikuti **Arrange, Act, Assert**, dan namanya adalah kalimat perilaku. **Fake** adalah implementasi tiruan dari interface yang sama, cepat, bisa diatur, dan bisa diperiksa. Fake bisa dipakai karena ViewModel menerima dependensinya lewat constructor.

ViewModel diuji dengan **runTest**, setelah `Dispatchers.Main` diganti dispatcher test, dan **advanceUntilIdle** menjalankan coroutine yang antre. Layar diuji dengan **runComposeUiTest** lewat pola cari, lakukan, periksa, dengan `testTag` untuk node penting.

**Dependency injection** berarti objek menerima dependensinya dari luar. **Koin** memindahkan pekerjaan merakit ke modul: `single` untuk satu objek bersama, `factory` untuk objek baru setiap kali, dan `viewModel` untuk ViewModel. Layar tidak berubah, hanya cara memperoleh ViewModel. Saat test, modul data diganti modul uji.

**Stack trace** dibaca dari atas untuk jenis error, turun ke `Caused by` untuk penyebab aslinya, lalu baris pertama dari kode sendiri sebagai tempat memperbaiki. Logcat disaring dengan filter seperti `package:mine` dan `is:crash`.

## PENUTUP

# Cek pemahaman

Jawab tanpa melihat modul dulu, lalu cocokkan dengan kunci di bawahnya.

1. Kenapa jumlah unit test seharusnya lebih banyak daripada UI test?
2. Apa itu fake, dan kenapa dipakai di unit test?
3. Kenapa ViewModel lebih mudah diuji daripada composable?
4. Apa yang terjadi kalau `advanceUntilIdle()` lupa dipanggil sebelum assert?
5. Apa bedanya single dan factory di Koin? Mana yang dipakai untuk Sumber, dan kenapa?
6. Apa yang berubah di layar saat dependensi disediakan Koin?
7. Dalam stack trace, di mana kalian mencari baris yang harus diperbaiki?

### JAWABAN Kunci jawaban

1. Unit test lebih cepat, lebih murah dijalankan, dan lebih mudah menunjuk baris yang salah, sedangkan UI test lambat dan mudah rusak saat tampilan berubah.
2. Implementasi tiruan dari interface yang sama, yang cepat dan bisa diatur, sehingga test tidak butuh jaringan atau kunci API.
3. ViewModel adalah kelas biasa tanpa UI. Cukup panggil fungsinya dan periksa state, tanpa menyalakan layar.
4. Coroutine di viewModelScope belum berjalan saat assert diperiksa, sehingga test gagal dengan nilai yang belum berubah.
5. single membuat satu objek untuk seluruh aplikasi, factory membuat objek baru setiap kali diminta. Sumber memakai single supaya semua layar berbagi data yang sama.
6. Isi layar tidak berubah. Yang berubah hanya cara memperoleh ViewModel, yaitu dengan koinViewModel.
7. Turun ke bagian Caused by yang paling bawah, lewati baris pustaka seperti java, lalu ambil baris pertama dari kode kita sendiri.

## PENUTUP

## Bacaan lanjut dan persiapan minggu depan

| Sumber                                                      | Untuk apa                               |
|-------------------------------------------------------------|-----------------------------------------|
| kotlinlang.org, Test your multiplatform app                 | Struktur commonTest dan kotlin.test     |
| kotlinlang.org, Testing Compose Multiplatform UI            | runComposeUiTest dan dependency ui-test |
| developer.android.com, Testing Kotlin coroutines on Android | runTest, setMain, dan advanceUntilIdle  |
| insert-koin.io, Koin for Compose                            | KoinApplication dan koinViewModel       |
| insert-koin.io, Koin Testing                                | KoinTest, startKoin, dan stopKoin       |
| developer.android.com, View logs with Logcat                | Filter Logcat                           |

**CONTOH Masalah Pertemuan 11**

Kode kelompok sudah besar dan dikerjakan bertiga. Bagaimana memastikan setiap push tetap bisa di-build dan semua test lulus, tanpa mengandalkan laptop salah satu anggota? Mulai Pertemuan 11 kita masuk fase proyek kelompok. Pastikan desktopTest lulus di laptop setiap anggota, dan bawa draf kebutuhan serta pembagian kerja kelompok untuk sprint 1.